

Proyecto de Título

Evaluación Parcial N°3

Estado de avance final del proyecto



Plataforma de reclutamiento laboral con inteligencia artificial

Integrantes:

Vicente Aravena

Ricardo Cuevas

Fabian Ortega

Profesor:

Cristian Marcelo Larrain Larrain

Ramo:

TALLER APLICADO DE PROGRAMACIÓN

Sección:

006V

Índice

1. Introducción del proyecto
2. Resumen Evaluación Parcial N°1
3. Resumen Evaluación Parcial N°2
4. Plan de pruebas del software
5. Casos de prueba utilizados.
 - CP 01 - CP 15
6. Base de datos de prueba utilizada
 - Fragmentos SQL utilizados como respaldo técnico
7. Evidencias de ejecución de pruebas
 - Implementación de tres grupos distintos de test
 - Pruebas realizadas en postman
 - Pantallazos de postman
8. Resultados obtenidos
9. Mejoras realizadas a partir de las pruebas aplicadas
10. Evidencias del producto final
 - Pantallazos más relevantes
11. Documentación de control de versiones
12. Configuraciones solicitadas en entregas anteriores
 - Respuesta aplicada en el MVP
 - Respaldo técnico en backend
 - Estructura de respuesta enviada al frontend
 - Respaldo técnico en frontend
 - Relación con la observación recibida
13. Documento de aceptación del cliente
14. Conclusión
15. Lecciones aprendidas

1.- Introducción

El presente informe tiene como finalidad documentar el estado final del proyecto **WorkSí**, este corresponde a una plataforma tecnológica orientada al área de reclutamiento laboral, cuyo objetivo es mejorar la búsqueda, postulación y selección de candidatos mediante una solución digital integrada. La problemática abordada surge a partir de las dificultades presentes en los procesos tradicionales de selección, tales como la revisión manual de currículums, la alta cantidad de postulaciones poco filtradas, la falta de transparencia en los criterios de evaluación y la posible presencia de sesgos durante las primeras etapas del reclutamiento.

Para responder a esta necesidad, el proyecto propone una solución compuesta por una **aplicación móvil para candidatos**, una **plataforma web para empresas y reclutadores**, un **backend centralizado** y un **servicio de inteligencia artificial**. A través de estos componentes, los candidatos pueden registrarse, crear su perfil, cargar su currículum, visualizar ofertas laborales y postular de forma sencilla. Por su parte, las empresas pueden gestionar ofertas, revisar postulantes y apoyarse en criterios de compatibilidad para facilitar la toma de decisiones.

Uno de los elementos principales de WorkSí es la incorporación de un sistema de **matching entre currículum y oferta laboral**, el cual permite calcular un porcentaje de compatibilidad entre el perfil del postulante y los requisitos de una vacante. Además, se consideran mecanismos de limpieza de datos sensibles, con el propósito de disminuir posibles sesgos y enfocar la evaluación en habilidades, experiencia y conocimientos relevantes para el cargo.

Durante la Evaluación Parcial N°1, el trabajo se enfocó en definir la problemática, los objetivos, el alcance, la planificación inicial y las tecnologías a utilizar. Posteriormente, en la Evaluación Parcial N°2, se avanzó en el diseño técnico del sistema, considerando arquitectura, modelo de datos, casos de uso, ambiente de desarrollo, implementación inicial y primeras evidencias funcionales.

En esta evaluación, el informe se centra en presentar el estado final del proyecto, incorporando el plan de pruebas, casos de prueba, evidencias de ejecución, resultados obtenidos, mejoras realizadas y evidencias del producto final. De esta manera, el documento busca demostrar que WorkSí responde a la problemática planteada, integrando componentes funcionales y técnicos que permiten validar la solución desarrollada dentro del alcance académico definido.

2. Resumen de la Evaluación Parcial 1

Durante la **Evaluación Parcial N°1** se desarrolló la base inicial del proyecto **WorkSí**, estableciendo la problemática, los objetivos, el alcance y la planificación general de la solución. En esta etapa se identificó como necesidad principal mejorar los procesos de reclutamiento laboral, los cuales muchas veces se realizan de forma manual, lenta y poco eficiente. Las empresas deben revisar una gran cantidad de currículums sin filtros precisos, mientras que los postulantes no siempre cuentan con información clara sobre su nivel de compatibilidad con una oferta laboral ni sobre el avance de sus postulaciones.

A partir de esta problemática, se propuso el desarrollo de una plataforma digital orientada a facilitar la conexión entre candidatos y empresas, incorporando tecnología para automatizar parte del proceso de selección. La solución fue planteada como un sistema compuesto por una aplicación móvil para candidatos, una plataforma web para empresas y reclutadores, un backend centralizado, una base de datos relacional y un servicio de inteligencia artificial encargado de apoyar el análisis de compatibilidad entre currículums y ofertas laborales.

En esta primera evaluación también se definieron los principales actores del sistema, considerando al candidato, reclutador, empresa y administrador. Cada uno de estos perfiles cumple un rol específico dentro de la plataforma: el candidato interactúa con las ofertas y gestiona su información personal y laboral; el reclutador administra publicaciones y revisa postulantes; la empresa mantiene sus datos institucionales; y el administrador gestiona elementos generales del sistema. Esta definición permitió ordenar las funcionalidades según las necesidades de cada usuario.

Junto con lo anterior, se establecieron las funcionalidades iniciales del Producto Mínimo Viable, tales como registro e inicio de sesión, creación de perfil del candidato, carga de currículum, publicación de ofertas laborales, visualización de ofertas y postulación a empleos. Estas funcionalidades fueron seleccionadas por representar el flujo principal del sistema, desde el ingreso del usuario hasta la interacción entre candidato y oferta laboral. De esta forma, se delimitó un alcance realista y alcanzable dentro del periodo académico disponible.

Finalmente, se realizó una planificación inicial del trabajo, considerando la organización del equipo, la distribución de tareas, las tecnologías a utilizar y los entregables esperados para las siguientes etapas. Esta evaluación permitió transformar una idea general en una propuesta formal de proyecto, estableciendo una base clara para avanzar posteriormente hacia el diseño técnico, la implementación de componentes, las pruebas funcionales y la validación final de WorkSí.

3. Resumen de la Evaluación Parcial 2

Durante la Evaluación Parcial N°2, el proyecto WorkSí avanzó desde una etapa de definición y planificación hacia una fase más técnica, enfocada en la arquitectura del sistema, las tecnologías utilizadas, la configuración del ambiente de trabajo y las primeras evidencias del avance funcional del MVP. A diferencia de la primera evaluación, donde se establecieron principalmente la problemática, los objetivos y el alcance, en esta segunda instancia se comenzó a consolidar la solución desde una perspectiva de desarrollo, integrando los distintos componentes necesarios para el funcionamiento del sistema.

Uno de los aspectos principales trabajados fue la arquitectura general de WorkSí, la cual se organizó en una aplicación móvil para candidatos, una plataforma web para administradores y reclutadores, un backend centralizado desarrollado en Spring Boot, una base de datos MySQL, un servicio de inteligencia artificial en FastAPI y almacenamiento de archivos en filesystem. Esta organización permitió separar responsabilidades entre los componentes del sistema, manteniendo una estructura clara para el desarrollo del Producto Mínimo Viable. El backend fue definido como el núcleo de comunicación, encargado de exponer la API REST, gestionar autenticación mediante JWT, aplicar reglas de negocio y coordinar la conexión con la base de datos y el servicio de inteligencia artificial.

También se documentaron los principales procesos funcionales del sistema, tales como el registro de candidatos, la publicación de ofertas laborales, el proceso de matching con inteligencia artificial, la postulación del candidato y la revisión de postulantes por parte del reclutador. Estos procesos permitieron representar de manera más concreta el flujo de trabajo de la plataforma, mostrando cómo interactúan los usuarios con el sistema y cómo se comunican los componentes internos para cumplir con las funcionalidades principales. En este sentido, la Evaluación Parcial N°2 permitió visualizar con mayor claridad el recorrido completo del usuario dentro de WorkSí, desde el registro hasta la postulación y revisión de compatibilidad.

Otro punto importante fue la definición y justificación de las tecnologías utilizadas. Para el backend se consideró Java 21 junto con Spring Boot 3.2.x, Spring Security, JPA/Hibernate, Flyway y Apache Tika. La plataforma web fue desarrollada con React, Vite y React Router, mientras que la aplicación móvil se planteó con Kotlin, Android Studio, Jetpack Compose, ViewModel y Retrofit. Por su parte, el servicio de inteligencia artificial se trabajó con Python 3.11, FastAPI, sentence-transformers, spaCy y scikit-learn, permitiendo realizar procesamiento de texto, generación de embeddings y cálculo de similitud semántica. Además, se incorporaron Docker y Docker Compose para facilitar la ejecución integrada de los servicios principales.

En esta evaluación también se configuró el ambiente de pruebas, buscando replicar de forma ordenada un entorno similar al de producción. Para ello, se consideró la ejecución conjunta del backend, la base de datos y el servicio de inteligencia artificial mediante contenedores separados. La aplicación web y la aplicación móvil consumen la misma API REST, permitiendo validar el comportamiento real del sistema mediante solicitudes HTTP, autenticación JWT y comunicación entre servicios. Asimismo, se utilizó Postman como herramienta de apoyo para comprobar endpoints relacionados con autenticación, registro, creación de empresas, publicación de ofertas, postulaciones e integración con el servicio de IA.

Finalmente, la Evaluación Parcial N°2 permitió evidenciar un avance significativo del MVP, ya que no solo se documentó la arquitectura propuesta, sino que también se mostraron componentes técnicos implementados y funcionando de manera integrada. Esta etapa fue clave para pasar desde la planificación inicial hacia una solución funcional en desarrollo, con tecnologías definidas, ambiente configurado, flujos principales identificados y primeras validaciones técnicas realizadas. De esta manera, se sentaron las bases necesarias para que en la Evaluación Parcial N°3 el foco pudiera centrarse en las pruebas finales, resultados obtenidos, mejoras aplicadas y evidencias del producto final.

4. Plan de pruebas del software

El plan de pruebas del proyecto WorkSí tuvo como objetivo validar que las funcionalidades principales del MVP respondieran correctamente desde dos enfoques complementarios: pruebas funcionales/manuales y pruebas automatizadas. En una primera etapa, el equipo validó el sistema ejecutando los servicios, revisando la plataforma web, probando la aplicación móvil desde Android Studio y utilizando Postman para comprobar endpoints del backend. Posteriormente, en la versión más reciente del Sprint 11, este enfoque se reforzó con tests unitarios, tests de integración y tests del servicio de inteligencia artificial.

El enfoque manual permitió revisar el comportamiento visible del sistema desde la perspectiva de los usuarios principales. Para esto se levantaron los servicios del proyecto, se ejecutó el frontend web mediante npm, se probó la aplicación móvil desde Android Studio y se realizaron solicitudes en Postman. Estas pruebas permitieron comprobar flujos como inicio de sesión, registro de candidatos, creación de empresas, gestión de reclutadores, publicación de ofertas, visualización de ofertas por parte del candidato, interacción con vacantes y revisión de postulantes.

A partir de la última versión del proyecto, el plan de pruebas se amplió incorporando un primer grupo de pruebas automatizadas en el backend. Este grupo corresponde a 11 tests unitarios Java, ubicados en el package service dentro de backend/src/test/java/cl/duoc/worksi/. Estos tests se ejecutan desde la carpeta producto/backend mediante el comando mvn test y validan componentes del sistema de manera aislada. Entre ellos se encuentran pruebas sobre ProductMatchServiceTest y ExperienceScoreUtilTest, enfocadas en el cálculo de compatibilidad, desglose del score, experiencia requerida, preferencias del candidato y casos borde.

El segundo grupo corresponde a 2 tests Java de integración, ubicados en el package integration. Estas pruebas se ejecutan con el perfil de integración mediante el comando mvn test -Pintegration, pudiendo utilizar el servicio de IA disponible en <http://localhost:8000> mediante la variable WORKSI_IT_AI_URL. A diferencia de los tests unitarios, estas pruebas validan una cadena más completa del sistema, incluyendo backend, base de datos mediante Testcontainers y servicio de inteligencia artificial. Su objetivo es comprobar que el cálculo del score y el desglose de compatibilidad funcionen correctamente en un flujo cercano al uso real.

El tercer grupo corresponde a 10 tests Python del servicio ai-service, ejecutados desde producto/ai-service mediante el comando pytest. Estos tests validan el comportamiento del servicio de inteligencia artificial, incluyendo el endpoint /health, validaciones de entrada,

endpoint /match, limpieza de información sensible, cálculo semántico y generación de explicaciones. De esta forma, se comprobó el comportamiento del componente de IA de manera independiente al backend.

En conjunto, el plan de pruebas permitió cubrir distintos niveles de validación. Las pruebas manuales y Postman permitieron revisar el uso funcional del MVP desde la experiencia real de ejecución, mientras que las pruebas automatizadas permitieron asegurar reglas internas, casos borde e integración entre componentes críticos. Esta combinación entrega una validación más sólida de WorkSí, ya que no se limita solo a observar que las pantallas funcionen, sino que también comprueba que la lógica principal del sistema responda correctamente.

En síntesis, el plan de pruebas evolucionó desde una estrategia principalmente manual y exploratoria hacia una validación más completa. La versión final del proyecto mantiene las pruebas realizadas mediante Postman, web y Android Studio, pero además incorpora pruebas automatizadas que respaldan especialmente el cálculo de compatibilidad, la integración con base de datos y el funcionamiento del servicio de inteligencia artificial.

5. Casos de prueba utilizados

Para complementar el plan de pruebas, se definieron casos de prueba orientados a validar los flujos más importantes del MVP de WorkSí. Estos casos fueron seleccionados a partir de las funcionalidades implementadas en el repositorio y del modo en que el equipo realizó la validación del sistema: pruebas manuales desde la interfaz web, ejecución de la aplicación móvil en Android Studio, revisión de endpoints mediante Postman y comprobación de la comunicación entre backend, base de datos y servicio de inteligencia artificial.

Los casos se organizaron considerando los roles principales del sistema: administrador, reclutador y candidato. Además, se incluyeron pruebas técnicas de ambiente e integración, ya que el correcto funcionamiento del proyecto depende de que los servicios principales se ejecuten de forma coordinada. En cada caso se verificó que la acción realizada produjera una respuesta coherente con el flujo esperado, ya fuera mediante una respuesta HTTP correcta, una actualización visible en pantalla o una navegación funcional dentro de la aplicación.

Módulo evaluado: Infraestructura general del sistema.

Objetivo: Verificar que los servicios principales del proyecto puedan iniciarse correctamente antes de ejecutar pruebas funcionales.

Acción realizada: Se levantó el ambiente considerando los servicios de MySQL, backend Spring Boot y servicio de inteligencia artificial. Posteriormente, se comprobó que el backend quedara disponible en el puerto 8080, que la base de datos respondiera en el puerto 3306 y que el servicio IA se encontrara activo en el puerto 8000.

Resultado esperado: Los contenedores o servicios deben iniciar sin errores críticos, el backend debe conectarse correctamente a MySQL y el servicio de IA debe quedar disponible para recibir solicitudes.

Resultado obtenido: El ambiente permitió ejecutar pruebas integradas del sistema. Esta validación fue importante porque confirmó que el backend podía comunicarse con la base de datos y con el servicio de inteligencia artificial, permitiendo continuar con las pruebas de usuarios, ofertas y postulaciones.

CP-02 | Inicio de sesión de usuario

Módulo evaluado: Autenticación y seguridad.

Objetivo: Validar que un usuario registrado pueda iniciar sesión correctamente y recibir acceso al sistema según su rol.

Acción realizada: Se probó el endpoint POST /api/v1/auth/login mediante Postman, enviando correo y contraseña válidos. También se verificó el flujo desde la pantalla de login de la plataforma web, observando que el usuario fuera redirigido a la vista correspondiente según su perfil.

Resultado esperado: El sistema debe validar las credenciales, generar una respuesta correcta y permitir el acceso únicamente si los datos son válidos. En caso de credenciales incorrectas, no debe permitir el ingreso.

Resultado obtenido: El inicio de sesión permitió comprobar el funcionamiento de la autenticación mediante JWT y la protección de rutas privadas. Esta prueba fue clave para validar que administradores y reclutadores pudieran acceder solo a las secciones que les correspondían.

CP-03 | Registro de candidato con carga de currículum

Módulo evaluado: Registro de candidato y almacenamiento de CV.

Objetivo: Comprobar que un candidato pueda registrarse en el sistema enviando sus datos personales y adjuntando su currículum en formato de archivo.

Acción realizada: Se probó el endpoint POST /api/v1/auth/register/candidate, enviando la información del candidato como data y el archivo del currículum como file mediante una solicitud multipart/form-data. Esta prueba permitió validar el ingreso de datos y la carga del documento asociado al postulante.

Resultado esperado: El sistema debe registrar al candidato, almacenar la referencia del archivo CV y dejar el perfil disponible para futuras consultas y postulaciones.

Resultado obtenido: La prueba permitió comprobar que el backend recibía correctamente datos y archivo en una misma solicitud. Además, permitió validar que el flujo de registro del candidato estuviera alineado con la necesidad principal del proyecto: utilizar el currículum como base para el proceso de matching laboral.

CP-04 | Consulta y actualización del perfil candidato

Módulo evaluado: Perfil de candidato.

Objetivo: Verificar que un candidato autenticado pueda consultar y actualizar información de su perfil.

Acción realizada: Se realizaron pruebas sobre los endpoints GET /api/v1/candidate/profile y PATCH /api/v1/candidate/profile. Primero se consultó la información del perfil y luego se modificaron algunos datos mediante una solicitud de actualización.

Resultado esperado: El sistema debe retornar la información asociada al usuario autenticado y permitir modificar solo los campos permitidos del perfil.

Resultado obtenido: La prueba permitió validar que la información del candidato se obtiene según el usuario autenticado y que la actualización mantiene la relación correcta entre cuenta de usuario y perfil laboral. Este caso fue importante para revisar que el candidato pueda mantener su información actualizada antes de postular.

CP-05 | Consulta de catálogos del sistema

Módulo evaluado: Catálogos generales.

Objetivo: Validar que la plataforma pueda entregar datos base para formularios y filtros, tales como regiones, comunas, sectores y habilidades.

Acción realizada: Se probaron los endpoints GET /api/v1/catalogs/regions, GET /api/v1/catalogs/regions/{region_id}/communes, GET /api/v1/catalogs/sectors y GET /api/v1/catalogs/sectors/{sector_id}/skills.

Resultado esperado: El sistema debe retornar listas de datos correctamente estructuradas para ser utilizadas por la plataforma web, la app móvil y los formularios del sistema.

Resultado obtenido: La consulta de catálogos permitió comprobar que el sistema contaba con datos de apoyo para completar registros, ofertas laborales y perfiles. También permitió validar casos de respuesta correcta y casos donde un sector no tuviera habilidades asociadas o no existiera.

CP-06 | Creación de empresa desde rol administrador

Módulo evaluado: Administración de empresas.

Objetivo: Verificar que el administrador pueda registrar una empresa dentro del sistema, incluyendo datos principales e imagen opcional.

Acción realizada: Se probó el endpoint POST /api/v1/admin/companies mediante una solicitud multipart/form-data. La prueba contempló el envío de los datos de empresa y, cuando correspondía, una imagen asociada.

Resultado esperado: El sistema debe crear la empresa correctamente y dejarla disponible para ser consultada, editada o asociada a reclutadores.

Resultado obtenido: Este caso permitió validar una de las funciones base del administrador, ya que antes de que un reclutador pueda operar correctamente dentro de la plataforma, debe existir una empresa registrada. La prueba también permitió revisar el manejo de solicitudes con archivos opcionales.

CP-07 | Creación y gestión de reclutador

Módulo evaluado: Administración de reclutadores.

Objetivo: Comprobar que el administrador pueda crear, consultar, modificar y eliminar usuarios reclutadores.

Acción realizada: Se probaron los endpoints asociados a reclutadores, principalmente POST /api/v1/admin/recruiters, GET /api/v1/admin/recruiters, GET /api/v1/admin/recruiters/{recruiter_user_id}, PATCH /api/v1/admin/recruiters/{recruiter_user_id} y DELETE /api/v1/admin/recruiters/{recruiter_user_id}.

Resultado esperado: El sistema debe permitir gestionar reclutadores de forma controlada, manteniendo la relación entre el usuario reclutador y la empresa correspondiente.

Resultado obtenido: La prueba permitió comprobar que el rol administrador tiene control sobre la creación y mantención de reclutadores. También permitió validar que la plataforma web protegiera estas pantallas para usuarios con rol ADMIN.

CP-08 | Creación de oferta laboral por reclutador

Módulo evaluado: Gestión de ofertas laborales.

Objetivo: Validar que un reclutador autenticado pueda crear una oferta laboral asociada a su empresa.

Acción realizada: Se probó el endpoint POST /api/v1/company/jobs mediante multipart/form-data, enviando los datos de la oferta y una imagen opcional. Además, se **revisó el flujo desde la plataforma web**, ingresando a la vista de creación de oferta laboral y comprobando que el formulario respondiera correctamente.

Resultado esperado: El sistema debe registrar la oferta laboral, asociarla a la empresa del reclutador y dejarla disponible para consulta o publicación según su estado.

Resultado obtenido: La prueba permitió validar que el reclutador pudiera avanzar desde la creación de una oferta hasta su posterior visualización en el listado. Este caso fue uno de los más relevantes, ya que representa el punto de partida para que los candidatos puedan encontrar vacantes dentro de la plataforma.

CP-09 | Listado, detalle y cambio de estado de ofertas

Módulo evaluado: Administración y seguimiento de ofertas.

Objetivo: Verificar que las ofertas creadas puedan ser listadas, consultadas en detalle, actualizadas y activadas o desactivadas según corresponda.

Acción realizada: Se probaron endpoints como GET /api/v1/company/jobs, GET /api/v1/company/jobs/{job_id}, PATCH /api/v1/company/jobs/{job_id}, PATCH /api/v1/company/jobs/{job_id}/status y DELETE /api/v1/company/jobs/{job_id}. Desde la interfaz web se revisó que el reclutador pudiera navegar entre el listado, detalle y edición de ofertas.

Resultado esperado: El sistema debe mostrar las ofertas asociadas al reclutador, permitir consultar su detalle y modificar su estado sin afectar ofertas de otras empresas.

Resultado obtenido: La prueba permitió comprobar que la gestión de ofertas funcionaba como flujo completo para el reclutador. Además, permitió validar que los cambios de estado fueran importantes para controlar qué ofertas se encuentran disponibles para los candidatos.

CP-10 | Visualización de ofertas desde candidato

Módulo evaluado: Feed de ofertas para candidatos.

Objetivo: Comprobar que el candidato pueda visualizar ofertas laborales activas y consultar el detalle de una oferta específica.

Acción realizada: Se probaron los endpoints GET /api/v1/candidate/jobs/feed y GET /api/v1/candidate/jobs/{job_id}. También se revisó el comportamiento desde la aplicación móvil, observando que las ofertas se cargaran y que la navegación hacia el detalle fuera coherente.

Resultado esperado: El sistema debe mostrar ofertas disponibles para el candidato, respetando paginación y entregando información suficiente para que el usuario decida si desea postular, guardar o descartar.

Resultado obtenido: Esta prueba permitió validar el flujo principal del candidato dentro de WorkSí. La correcta visualización de ofertas es fundamental, ya que conecta la información publicada por empresas con la experiencia móvil del postulante.

CP-11 | Swipe, guardado y descarte de ofertas

Módulo evaluado: Interacción candidato-oferta.

Objetivo: Validar que el candidato pueda interactuar con las ofertas mediante acciones similares a guardar, descartar o postular.

Acción realizada: Se probó el endpoint POST /api/v1/candidate/swipes para registrar la interacción del candidato con una oferta. Además, se revisaron los endpoints GET /api/v1/candidate/saved-jobs, POST /api/v1/candidate/saved-jobs y DELETE /api/v1/candidate/saved-jobs/{job_id} para comprobar el guardado y eliminación de ofertas favoritas.

Resultado esperado: El sistema debe registrar correctamente la acción realizada por el candidato y reflejarla en las consultas posteriores, evitando inconsistencias en el historial de interacción.

Resultado obtenido: La prueba permitió validar la funcionalidad característica del proyecto, donde el candidato no solo revisa ofertas, sino que interactúa con ellas de forma simple y rápida. Esto refuerza la propuesta de una experiencia más dinámica que los portales tradicionales de empleo.

CP-12 | Revisión de postulantes por reclutador

Módulo evaluado: Gestión de postulaciones.

Objetivo: Verificar que el reclutador pueda revisar postulaciones asociadas a una oferta laboral y consultar información relevante del candidato.

Acción realizada: Se probaron endpoints como GET /api/v1/company/jobs/{job_id}/applications, GET /api/v1/company/jobs/{job_id}/applications/{application_id}, GET /api/v1/company/jobs/{job_id}/applications/{application_id}/candidate-profile y GET /api/v1/company/jobs/{job_id}/applications/{application_id}/cv. También se revisó la vista web de postulaciones, donde el reclutador puede acceder al detalle de una postulación.

Resultado esperado: El sistema debe listar los postulantes de una oferta, mostrar información relevante del candidato y permitir consultar datos asociados a su currículum.

Resultado obtenido: Esta prueba permitió comprobar la utilidad del sistema para el reclutador, ya que no solo se crean ofertas, sino que también se puede hacer seguimiento de los candidatos interesados. Además, permitió validar que la información se presentara de forma organizada para apoyar la toma de decisiones.

CP-13 | Validación del servicio de inteligencia artificial

Módulo evaluado: Servicio IA y cálculo de compatibilidad.

Objetivo: Comprobar que el servicio de inteligencia artificial responda correctamente al recibir texto de un currículum y texto de una oferta laboral.

Acción realizada: Se probó el endpoint GET /health para verificar disponibilidad del servicio y el endpoint POST /match enviando cv_text y job_text. También se probaron casos inválidos, como textos vacíos o demasiado cortos.

Resultado esperado: El servicio debe responder con estado UP en la prueba de salud y, en el caso de matching, debe retornar un score de compatibilidad junto con una explicación. Si el contenido no es suficiente, debe responder con un error controlado.

Resultado obtenido: La prueba permitió validar que el módulo de IA no solo calculaba compatibilidad, sino que también aplicaba controles sobre el contenido recibido. Este caso fue importante porque el matching es uno de los elementos diferenciadores de WorkSí frente a una plataforma tradicional de empleo.

CP-14 | Navegación protegida en plataforma web

Módulo evaluado: Frontend web y control de acceso por rol.

Objetivo: Verificar que las rutas de la plataforma web respondan según el rol del usuario autenticado.

Acción realizada: Se ejecutó el frontend mediante npm run dev y se revisaron rutas como /home, /companies, /recruiters, /ofertas, /recruiter/home, /recruiter/ofertas y /recruiter/matches. Se validó que las rutas de administrador estuvieran protegidas para usuarios ADMIN y que las rutas de reclutador estuvieran protegidas para usuarios RECRUITER.

Resultado esperado: El sistema debe permitir el acceso solo a las vistas correspondientes al rol del usuario y redirigir o bloquear el acceso cuando no corresponda.

Resultado obtenido: La prueba permitió revisar que la interfaz web no fuera solo visual, sino que también respetara la separación de responsabilidades entre administrador y reclutador. Esta validación complementó las pruebas realizadas en backend mediante JWT.

CP-15 | Ejecución funcional de la aplicación móvil

Módulo evaluado: Aplicación móvil Android.

Objetivo: Validar que la aplicación móvil permitiera al candidato recorrer el flujo principal de visualización e interacción con ofertas laborales.

Acción realizada: Se ejecutó la aplicación desde Android Studio, revisando que las pantallas cargaran correctamente, que la navegación fuera coherente y que las acciones del candidato se reflejaran según lo esperado. Se puso especial atención en el inicio de sesión, visualización de ofertas, detalle de oferta e interacción del usuario con las vacantes.

Resultado esperado: La aplicación debe iniciar correctamente, conectarse con el backend configurado, mostrar información al candidato y permitir acciones propias del flujo móvil.

Resultado obtenido: La ejecución manual desde Android Studio permitió validar el comportamiento general de la app desde la perspectiva del usuario final. Esta prueba fue especialmente útil para detectar detalles visuales, problemas de navegación o respuestas inesperadas que no siempre se observan desde Postman.

6. Base de datos de prueba utilizada

Para la ejecución de las pruebas del proyecto WorkSí se utilizó una base de datos MySQL asociada al ambiente de desarrollo y validación del MVP. Esta base permitió almacenar información necesaria para probar los flujos principales del sistema, tales como autenticación,

perfiles de candidatos, empresas, reclutadores, ofertas laborales, currículums, postulaciones, acciones de swipe y resultados de compatibilidad.

En las pruebas manuales y funcionales, la base de datos correspondió a la base worksi levantada junto con los demás servicios del proyecto. Esta fue utilizada al ejecutar el backend, la plataforma web, la aplicación móvil y las solicitudes desde Postman, permitiendo comprobar que las acciones realizadas desde las distintas interfaces persistieran correctamente la información.

Además, con la incorporación de tests de integración en el backend, se agregó un segundo enfoque de base de datos de prueba: el uso de Testcontainers. En este caso, las pruebas levantan una base MySQL temporal con nombre worksi, usuario worksi y contraseña worksi, permitiendo ejecutar escenarios controlados sin depender directamente de los datos manuales del ambiente local. Esto es especialmente útil para validar la integración del backend con la base de datos de forma repetible.

Dentro de las pruebas de integración se generaron datos específicos para simular un escenario real de matching. Entre estos datos se incluyen un usuario candidato, perfil del candidato, preferencias de modalidad y jornada, currículum con texto normalizado, una empresa de prueba y dos ofertas laborales: una alineada al perfil del candidato y otra no relacionada. Esta preparación permitió comprobar que el sistema asignara mayor compatibilidad a la oferta que realmente coincidía con el perfil laboral.

La estructura utilizada en las pruebas se apoyó en tablas como users, candidate_profiles, candidate_cvs, companies, jobs, applications, candidate_job_swipes y saved_jobs. Estas tablas permitieron validar relaciones importantes del sistema, por ejemplo, que una oferta pertenezca a una empresa, que un currículum pertenezca a un candidato y que una postulación relacione correctamente candidato y oferta.

En síntesis, la base de datos de prueba cumplió un rol central en la validación del proyecto. En el flujo manual permitió comprobar persistencia y coherencia de datos durante el uso del MVP, mientras que en los tests de integración permitió validar la comunicación backend-base de datos en un entorno controlado y automatizado.

Fragmentos SQL utilizados como respaldo técnico

Para complementar la descripción anterior, se incorporan algunos fragmentos reales de la migración SQL utilizada en el backend. Estos fragmentos permiten evidenciar que la base de datos de prueba no fue una estructura improvisada, sino que responde a un esquema relacional definido mediante migraciones versionadas. A partir de estas tablas fue posible validar usuarios, perfiles, currículums, ofertas, postulaciones, acciones de swipe y resultados de compatibilidad.

Fragmento 1: tabla de usuarios y control de roles

```
CREATE TABLE users (  
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  role ENUM('CANDIDATE','ADMIN','RECRUITER') NOT NULL,  
  email VARCHAR(190) NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  is_active TINYINT(1) NOT NULL DEFAULT 1,  
  failed_login_attempts SMALLINT UNSIGNED NOT NULL DEFAULT 0,  
  lock_until DATETIME NULL,  
  last_login_at DATETIME NULL,  
  PRIMARY KEY (id),  
  UNIQUE KEY uk_users_email (email),  
  KEY idx_users_role (role),  
  KEY idx_users_active (is_active)  
) ENGINE=InnoDB;
```

Este fragmento respalda las pruebas relacionadas con autenticación, roles y control de acceso. La tabla users permite distinguir candidatos, administradores y reclutadores mediante el campo role, además de asegurar que el correo del usuario sea único dentro del sistema.

Fragmento 2: tabla de currículums del candidato

```
CREATE TABLE candidate_cvs (  
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  candidate_user_id BIGINT UNSIGNED NOT NULL,  
  original_filename VARCHAR(255) NOT NULL,  
  storage_path VARCHAR(700) NOT NULL,  
  file_size_bytes INT UNSIGNED NOT NULL,  
  mime_type VARCHAR(100) NOT NULL DEFAULT 'application/pdf',
```

```

extracted_text MEDIUMTEXT NULL,
normalized_text MEDIUMTEXT NULL,
is_current TINYINT(1) NOT NULL DEFAULT 1,
uploaded_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (id),
KEY idx_candidate_cvs_candidate (candidate_user_id),
CONSTRAINT fk_candidate_cvs_candidate
  FOREIGN KEY (candidate_user_id) REFERENCES candidate_profiles(user_id)
  ON UPDATE CASCADE
  ON DELETE CASCADE,
CONSTRAINT chk_candidate_cvs_pdf
  CHECK (LOWER(mime_type) = 'application/pdf'),
CONSTRAINT chk_candidate_cvs_max_1mb
  CHECK (file_size_bytes <= 1048576)
) ENGINE=InnoDB;

```

Este fragmento respalda las pruebas de registro de candidato y carga de currículum. La tabla no solo guarda la referencia del archivo, sino también el texto extraído y normalizado, lo cual es importante para el proceso de análisis y matching. Además, se establecen restricciones para aceptar archivos PDF y controlar el tamaño máximo permitido.

Fragmento 3: tabla de ofertas laborales

```

CREATE TABLE jobs (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  company_id BIGINT UNSIGNED NOT NULL,
  company_commercial_name VARCHAR(180) NOT NULL,
  title VARCHAR(180) NOT NULL,
  description TEXT NOT NULL,
  city VARCHAR(120) NOT NULL,
  region_id BIGINT UNSIGNED NOT NULL,
  commune_id BIGINT UNSIGNED NOT NULL,
  salary_offered INT UNSIGNED NOT NULL,
  years_experience_required TINYINT UNSIGNED NOT NULL,
  modality ENUM('REMOTE','HYBRID','ONSITE') NOT NULL,
  workload ENUM('FULL_TIME','PART_TIME','OTHER') NOT NULL,
  status ENUM('ACTIVE') NOT NULL DEFAULT 'ACTIVE',

```

```

PRIMARY KEY (id),
KEY idx_jobs_company (company_id),
KEY idx_jobs_status (status),
KEY idx_jobs_modality (modality),
KEY idx_jobs_workload (workload)
) ENGINE=InnoDB;

```

Este fragmento complementa la descripción de las pruebas sobre creación y gestión de ofertas laborales. La tabla jobs almacena los principales datos utilizados por el reclutador y, al mismo tiempo, contiene campos relevantes para el matching, como título, descripción, modalidad, jornada y años de experiencia requeridos.

Fragmento 4: tabla de postulaciones y resultado del match

```

CREATE TABLE applications (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  candidate_user_id BIGINT UNSIGNED NOT NULL,
  job_id BIGINT UNSIGNED NOT NULL,
  status ENUM('APPLIED','VIEWED','CANCELLED') NOT NULL DEFAULT 'APPLIED',
  match_score DECIMAL(5,2) NULL,
  match_explanation VARCHAR(500) NULL,
  matched_at DATETIME NULL,
  applied_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  UNIQUE KEY uk_applications_candidate_job (candidate_user_id, job_id),
  KEY idx_applications_job (job_id),
  KEY idx_applications_candidate (candidate_user_id),
  KEY idx_applications_job_score (job_id, match_score),
  CONSTRAINT fk_applications_candidate
    FOREIGN KEY (candidate_user_id) REFERENCES candidate_profiles(user_id)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
  CONSTRAINT fk_applications_job
    FOREIGN KEY (job_id) REFERENCES jobs(id)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
  CONSTRAINT chk_applications_score_range

```

```
CHECK (match_score IS NULL OR (match_score >= 0.00 AND match_score <= 100.00))  
) ENGINE=InnoDB;
```

Este fragmento respalda directamente las pruebas relacionadas con postulaciones y cálculo de compatibilidad. La restricción UNIQUE KEY uk_applications_candidate_job evita que un mismo candidato postule más de una vez a la misma oferta, mientras que el campo match_score permite almacenar el porcentaje de compatibilidad obtenido. Además, la restricción chk_applications_score_range asegura que el resultado del match se mantenga dentro de un rango válido entre 0 y 100.

Fragmento 5: acciones de swipe y ofertas guardadas

```
CREATE TABLE candidate_job_swipes (  
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  candidate_user_id BIGINT UNSIGNED NOT NULL,  
  job_id BIGINT UNSIGNED NOT NULL,  
  action ENUM('PASS','APPLY') NOT NULL,  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (id),  
  UNIQUE KEY uk_candidate_job_swipes_unique (candidate_user_id, job_id)  
) ENGINE=InnoDB;
```

```
CREATE TABLE saved_jobs (  
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  candidate_user_id BIGINT UNSIGNED NOT NULL,  
  job_id BIGINT UNSIGNED NOT NULL,  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (id),  
  UNIQUE KEY uk_saved_jobs_candidate_job (candidate_user_id, job_id)  
) ENGINE=InnoDB;
```

Estos fragmentos respaldan las pruebas asociadas a la experiencia principal del candidato. La tabla candidate_job_swipes registra si el candidato descarta o postula a una oferta, mientras que saved_jobs permite almacenar ofertas guardadas. En ambos casos se utilizan claves únicas para evitar registros duplicados entre un mismo candidato y una misma oferta.

En conjunto, estos fragmentos SQL complementan la descripción funcional de la base de datos de prueba. Además de permitir la persistencia de información, el esquema incorpora relaciones, claves únicas, índices y restricciones que ayudan a mantener la consistencia de los datos durante las pruebas realizadas sobre el MVP.

7. Evidencias de ejecución de pruebas

Implementación de tres grupos distintos de test:

Grupo 1: 11 Test java unitarios (miden el correcto funcionamiento de todos los elementos del sistema, pero por separado, no trabajando en conjunto):

Packages o archivos creados:

- En carpeta /backend/src/test/java/cl/duoc/worksi/, se creó package /service
- Archivos dentro de ese package nuevo: ProductMatchServiceTest.java, ExperienceScoreUtilTest.java

Ejecución:

```
cd "...\\producto\\backend"
```

```
mvn test
```

Respuesta exitosa:

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running cl.duoc.worksi.service.ExperienceScoreUtilTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.035 s -- in cl.duoc.worksi.service.ExperienceScoreUtilTest
[INFO] Running cl.duoc.worksi.service.ProductMatchServiceTest
WARNING: A Java agent has been loaded dynamically (C:\Users\VICENTE\.m2\repository\net\bytebuddy\byte-buddy-agent\1.14.13\byte-buddy-agent-1.14.13.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
Standard Commons Logging discovery in action with spring-jcl: please remove commons-logging.jar from classpath in order to avoid potential conflicts
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.236 s -- in cl.duoc.worksi.service.ProductMatchServiceTest
[INFO] Results:
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.573 s
[INFO] Finished at: 2026-06-16T15:13:49-04:00
[INFO] -----
```

(3 tests de service.ExperienceScoreUtilTest corridos con éxito – 8 tests de service.ProductMatchServiceTest corridos con éxito).

Grupo 2: 2 Tests Java Integración (Miden el funcionamiento end to end, es decir, de principio a fin en la cadena de todas las dimensiones del proyecto, en otras palabras, miden el trabajo exitoso y colaborativo entre todos los elementos: ia, backend, bd) en este caso, un test de integración mide el score total (desde que se sube cv, se limpia el texto, la ia lo limpia aun más, la transformación a vectores, el calculo de score final), y el otro test mide la respuesta del score breakdown (el desglose detallado para los usuarios de cómo cada dimensión tomada en cuenta para el score influye en el score final, la puntuación de cada dimensión), es decir, todo lo relacionado a cálculos de scores en el sistema está 100% ok y testeado.

Packages o archivos creados:

- Dentro de /backend/src/test/java/cl/duoc/worksi/ se creó package /integration
- Archivos dentro de ese package: IntegrationTestSupport.java, MatchScoreIntegrationTest.java

Ejecución:

```
cd "...\\producto\\backend"
$env:WORKSI_IT_AI_URL = "http://localhost:8000"
mvn test -Pintegration
```

Respuesta exitosa:

```
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 36.44 s -- in cl.duoc.worksi.integration.MatchScoreIntegrationTest
[INFO] Results:
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 40.465 s
[INFO] Finished at: 2026-06-16T15:27:01-04:00
[INFO] -----
```

Los 2 tests de “integration.MatchScoreIntegrationTest” salieron exitosos.
Esto confirma que todo esto funciona 100% bien:

- Backend + BD (Testcontainers)
- IA en localhost:8000
- Detalle de oferta con score + 5 dimensiones coherentes
- Oferta más alineada al perfil del candidato puntúa mayor que la no alineada

Grupo 3: 10 Tests de Python (servicio de IA):

Packages o archivos creados:

- Dentro de producto/ai-service/tests/ à se crearon 5 archivos nuevos -> test_fairness.py, test_main_fast.py, test_semantic_match.py, pytest.ini, requirements-dev.txt

Ejecución:

```
cd "...\\producto\\ai-service"
```

pytest

Respuesta exitosa:

```

PS C:\Users\VICENTE\Desktop\IT\Duoc\Duoc 5 Actual\worksi\producto\ai-service> pytest
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.1.0, pluggy-1.6.0
rootdir: C:\Users\VICENTE\Desktop\IT\Duoc\Duoc 5 Actual\worksi\producto\ai-service
configfile: pytest.ini
plugins: anyio-4.14.0
collected 10 items

tests\test_fairness.py ..
tests\test_main_fast.py .... [ 20%]
tests\test_semantic_match.py .... [ 60%]
                                     [100%]

===== warnings summary =====
tests/test_main_fast.py::test_match_returns_score_and_explanation
  C:\Users\VICENTE\AppData\Local\Programs\Python\Python311\Lib\site-packages\spacy\cli_util.py:
  23: DeprecationWarning: Importing 'parser.split_arg_string' is deprecated, it will only be avail
  able in 'shell_completion' in Click 9.0.
    from click.parser import split_arg_string

tests/test_semantic_match.py::test_semantic_score_within_bounds_for_aligned_pair
  C:\Users\VICENTE\AppData\Local\Programs\Python\Python311\Lib\site-packages\huggingface_hub\fil
  e_download.py:143: UserWarning: `huggingface_hub` cache-system uses symlinks by default to effi
  ciently store duplicated files but your machine does not support them in C:\Users\VICENTE\.cache\
  huggingface\hub\models--sentence-transformers--all-MiniLM-L6-v2. Caching files will still work b
  ut in a degraded version that might require more space on your disk. This warning can be disable
  d by setting the `HF_HUB_DISABLE_SYMLINKS_WARNING` environment variable. For more details, see h
  ttps://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.
  To support symlinks on Windows, you either need to activate Developer Mode or to run Python as
  an administrator. In order to activate developer mode, see this article: https://docs.microsoft
  .com/en-us/windows/apps/get-started/enable-your-device-for-development
    warnings.warn(message)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 10 passed, 2 warnings in 25.28s =====

```

Que midió cada test de Python para ai-service:

test_fairness.py (2 tests) à Limpieza de texto en ofertas/CV

test_main_fast.py (4 tests) à /health, validaciones y /match (IA mockeada)

test_semantic_match.py (4 tests) à Semántica real, fairness con spaCy, explicaciones del score

En resumen, estos grupos de test:

Tests Java unitarios -> testean fórmula + reglas + casos borde (escenarios sin CV, escenarios con IA caída...) pero no miden integración BD ni HTTP.

Tests Java integración -> Testean cadena real completa, hasta el JSON del API (lo que consume el frontend, es decir, el resultado de todo el proceso)

Tests Python en servicio ai-service -> Testean servicio de IA (fairness -> limpieza de entidades y palabras sensibles, transformación a vectores numéricos, etc) y limpieza de texto del CV. Testea esas dos cosas de forma aislada.

Pruebas realizadas en postman:

La siguiente tabla muestra las pruebas desarrolladas durante la anterior evaluación, estas consisten en la utilización de Postman para ver si los endpoints del backend están respondiendo correctamente.

Prueba	Endpoint / acción	Datos o condición probada	Resultado esperado	Resultado obtenido
Verificar disponibilidad del backend	GET /health	Backend ejecutándose en Docker	El sistema responde estado activo	Respuesta correcta con estado UP
Inicio de sesión válido	POST /api/v1/auth/login	Credenciales correctas de administrador o reclutador	Generación de token JWT y datos del usuario	Login exitoso y token recibido
Inicio de sesión inválido	POST /api/v1/auth/login	Credenciales incorrectas	Mensaje de error y rechazo de acceso	El sistema rechaza el login
Validación de rutas protegidas	GET /api/v1/admin/companies	Acceso con token ADMIN	Permitir acceso al listado de empresas	Acceso autorizado correctamente
Validación de permisos por rol	GET /api/v1/admin/companies	Acceso con token RECRUITER	Denegar acceso por rol incorrecto	El sistema bloquea el acceso
Consulta de catálogos	GET /api/v1/catalogs/regions	Solicitud pública sin token	Retornar listado de regiones	Catálogo cargado correctamente

Registro de reclutador	POST /api/v1/admin/recruiters	Datos válidos de reclutador y empresa asociada	Crear usuario reclutador	Reclutador registrado en base de datos
Consulta de perfil de empresa	GET /api/v1/company/profile	Token de reclutador válido	Mostrar datos de la empresa asociada	Perfil cargado correctamente
Creación de oferta laboral	POST /api/v1/company/jobs	Datos de oferta, skills e imagen opcional	Registrar oferta laboral	Oferta creada correctamente
Listado de ofertas del reclutador	GET /api/v1/company/jobs	Token de reclutador válido	Mostrar ofertas publicadas por la empresa	Listado obtenido correctamente
Feed de ofertas para candidato	GET /api/v1/candidate/jobs/feed	Token de candidato válido	Mostrar ofertas activas disponibles	Feed cargado desde backend
Recuperación de contraseña	POST /api/v1/auth/password-recovery/request	Correo registrado	Iniciar flujo de recuperación	Solicitud procesada correctamente
Verificación de recuperación	POST /api/v1/auth/password-recovery/verify	Código de recuperación MVP	Generar token de recuperación	Token generado correctamente

Cambio de contraseña	POST /api/v1/auth/password-recovery/reset	Nueva contraseña válida	Actualizar contraseña del usuario	Contraseña modificada correctamente
----------------------	--	-------------------------	-----------------------------------	-------------------------------------

Algunos pantallazos de lo visto en esta tabla:

GET

http://localhost:8000/openapi.json

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Query Params

Key	Value	Description
Key	Value	Description

Body

Cookies

Headers (4)

Test Results

200 OK

JSON

Preview

Visualize

```
1  {
2    "openapi": "3.1.0",
3    "info": {
4      "title": "FastAPI",
5      "version": "0.1.0"
6    },
7    "paths": {
8      "/health": {
9        "get": {
10          "summary": "Health",
11          "operationId": "health_health_get",
12          "responses": {
13            "200": {
14              "description": "Successful Response",
15              "content": {
16                "application/json": {
17                  "schema": {}
18                }
19            }
20          }
21        }
22      }
23    }
24  }
```

GET http://localhost:8080/api/v1/company/jobs

Docs Params Authorization Headers (7) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (14) Test Results 401 Unauthorized 66 ms 537 B

{ } JSON Preview Pass the correct auth credentials

```
1 {
2   "error": {
3     "message": "Se requiere autenticacion",
4     "trace_id": "",
5     "code": "UNAUTHORIZED",
6     "details": []
7   }
8 }
```

POST http://localhost:8080/api/v1/admin/recruiters

Docs Params Authorization Headers (10) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "email": "reclutador2@empresa.cl",
3   "password": "Reclutador123*",
4   "first_name": "Pedro",
5   "last_name_paternal": "Gonzalez",
6   "last_name_maternal": "Rojas",
7   "rut": "22222222-2",
8   "phone": "123456789",
9   "mobile": "987654321",
10  "birth_date": "1994-01-01",
11  "company_id": 1
12 }
```

Body Cookies Headers (14) Test Results 201 Created 2.53 s 508 B

{ } JSON Preview Visualize

```
1 {
2   "user_id": 5,
3   "role": "RECRUITER",
4   "email": "reclutador2@empresa.cl",
5   "company_id": 1
6 }
```

GET http://localhost:8080/api/v1/admin/companies

Docs Params **Authorization** Headers (8) Body Scripts Settings

Auth Type
Bearer Token

Token

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body Cookies Headers (14) Test Results

200 OK • 2.12 s • 751 B

JSON Preview Visualize

```
1 {
2   "items": [
3     {
4       "company_id": 2,
5       "commercial_name": "Chilexpress",
6       "legal_name": "Curier",
7       "rut": "96756430-3",
8       "corporate_email": "chilexpress@chilexpress.cl"
9     },
10    {
11      "company_id": 1,
12      "commercial_name": "Ricarditos",
13      "legal_name": "Venta",
14      "rut": "20327548-K",
15      "corporate_email": "randescri12@gmail.com"
16    }
17  ],
18  "page": 1,
19 }
```

8. Resultados obtenidos

A partir de las pruebas realizadas sobre WorkSí, se obtuvo como resultado general una validación más completa del MVP. En una primera etapa, las pruebas manuales permitieron comprobar que el ambiente podía levantarse correctamente, que el backend respondía a las solicitudes principales, que la plataforma web y la aplicación móvil permitían recorrer los flujos esperados, y que la base de datos almacenaba la información generada durante las pruebas funcionales.

Posteriormente, con la incorporación de pruebas automatizadas en el Sprint 11, los resultados obtenidos se fortalecieron desde el punto de vista técnico. En el backend se ejecutaron 11 tests unitarios Java, distribuidos entre `ProductMatchServiceTest` y `ExperienceScoreUtilTest`. Estos permitieron validar la fórmula de cálculo de compatibilidad, el desglose del score, la experiencia requerida, las preferencias de modalidad y jornada, además de casos borde relacionados con CV inexistente, texto no utilizable o servicio de IA no disponible.

Además, se ejecutaron 2 tests de integración Java mediante el perfil de Maven para integración. Estas pruebas validaron un flujo más cercano al comportamiento real del sistema, incorporando backend, base de datos mediante Testcontainers y servicio de inteligencia artificial. Como resultado, se confirmó que el detalle de una oferta retorna score y desglose de cinco dimensiones, y que una oferta alineada al perfil del candidato obtiene un puntaje superior a una oferta no relacionada.

En el servicio de inteligencia artificial también se obtuvieron resultados positivos. Se ejecutaron 10 tests Python con pytest, distribuidos en pruebas de fairness, pruebas rápidas del endpoint `/health` y `/match`, y pruebas de matching semántico. Estos resultados permitieron confirmar que el servicio responde correctamente, rechaza entradas insuficientes, limpia información sensible y mantiene coherencia al comparar un currículum con ofertas relacionadas o no relacionadas.

En síntesis, los resultados obtenidos evidencian que WorkSí no solo fue validado mediante pruebas manuales, Postman y ejecución visual de sus interfaces, sino también mediante pruebas automatizadas aplicadas a componentes críticos del sistema. Esto entrega mayor respaldo a la versión final del MVP, especialmente en el cálculo de compatibilidad, la integración con la base de datos y el funcionamiento del servicio de inteligencia artificial.

9. Mejoras realizadas a partir de las pruebas aplicadas

Las pruebas aplicadas durante el desarrollo de WorkSí cumplieron inicialmente un rol de verificación continua. En las primeras etapas, el equipo validó el funcionamiento del sistema principalmente mediante ejecución manual, revisión de pantallas, pruebas con Postman, levantamiento de servicios y comprobación progresiva de que cada componente permitiera avanzar al siguiente flujo del MVP.

Con el avance final del Sprint 11, esta estrategia fue reforzada mediante la incorporación de pruebas automatizadas. Esta mejora no modificó el enfoque general del proyecto ni implicó rediseñar los casos funcionales ya definidos, pero sí permitió respaldar técnicamente

componentes críticos que antes se validaban principalmente de forma manual. En especial, se fortaleció la validación del cálculo de compatibilidad, la integración entre backend, base de datos y servicio de inteligencia artificial, y el comportamiento interno del servicio ai-service.

Una de las mejoras más relevantes fue la incorporación de tests unitarios Java en el backend. Estos permitieron comprobar reglas específicas del sistema de matching, como el cálculo del score final, la influencia de modalidad, jornada y experiencia, además de escenarios borde como ausencia de CV, texto no utilizable o servicio de IA no disponible. Con esto, parte de la lógica principal dejó de depender únicamente de la revisión manual y pasó a contar con validación automatizada.

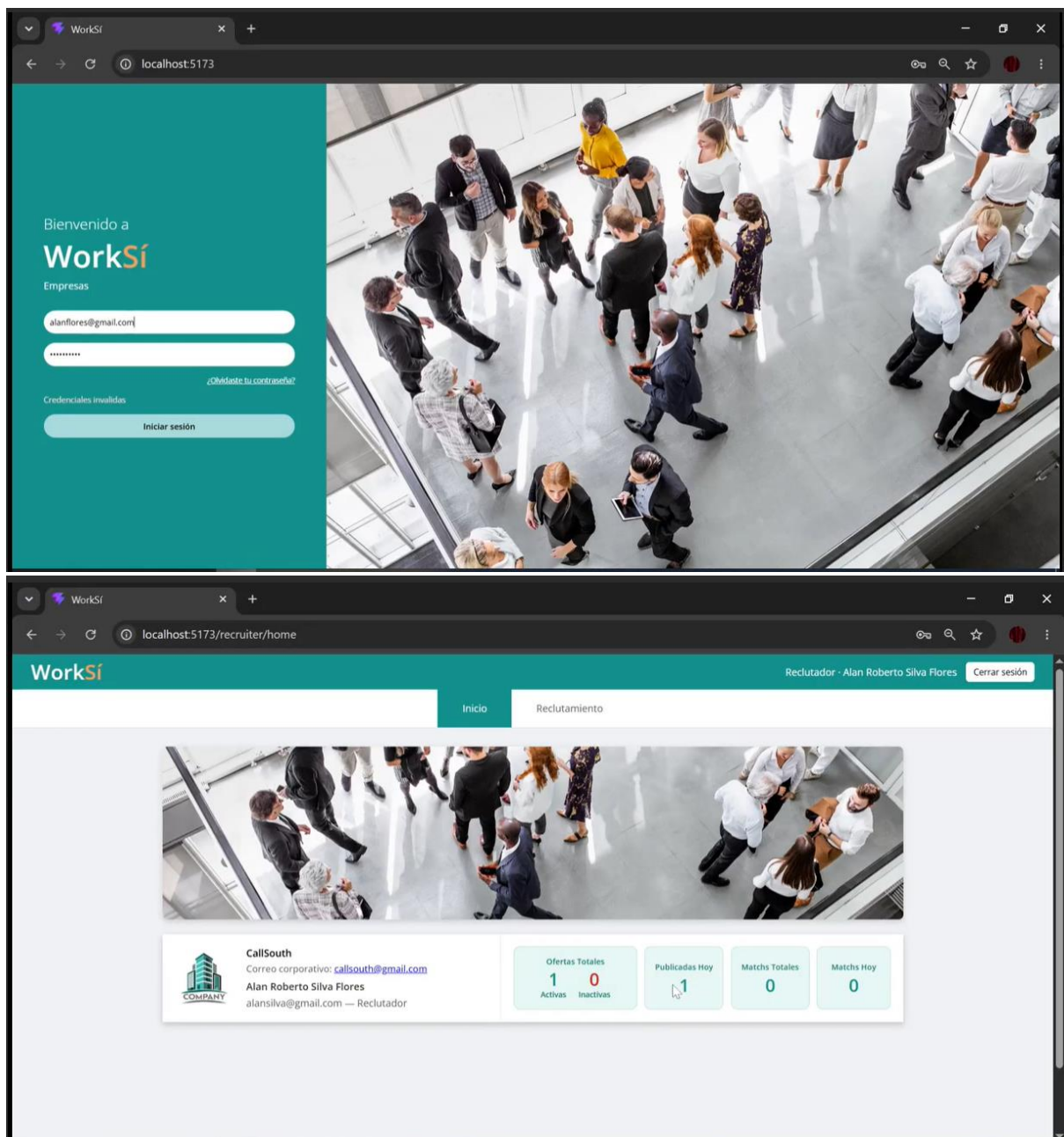
También se agregaron tests de integración, los cuales permitieron revisar el comportamiento de una cadena más completa del sistema. A diferencia de los tests unitarios, estas pruebas validan la interacción entre backend, base de datos de prueba mediante Testcontainers y servicio de inteligencia artificial. Gracias a esto se pudo comprobar que una oferta alineada al perfil del candidato obtiene un puntaje mayor que una oferta no relacionada, además de validar el desglose de las cinco dimensiones del score.

Finalmente, el servicio de inteligencia artificial incorporó pruebas en Python con pytest. Estas pruebas permiten validar el endpoint /health, el endpoint /match, el rechazo de datos insuficientes, la limpieza de información sensible y el cálculo semántico entre currículum y oferta laboral. De esta forma, la mejora aplicada permitió cubrir tanto la lógica del backend como el comportamiento propio del servicio de IA.

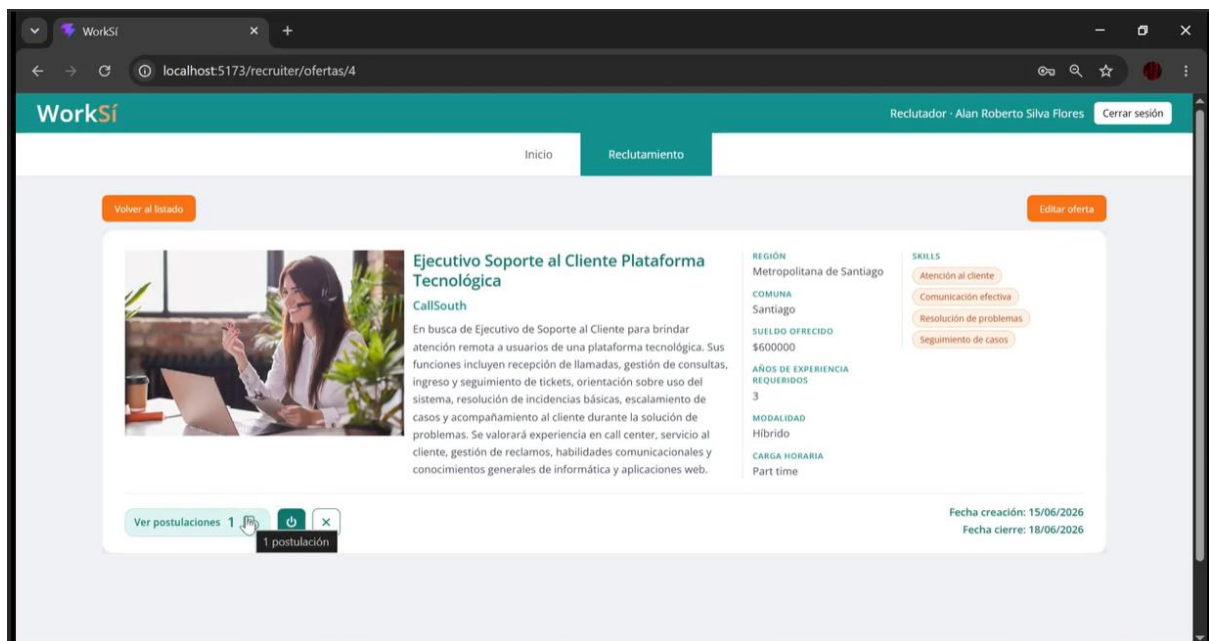
En síntesis, las mejoras realizadas a partir de las pruebas no consistieron en cambiar el alcance del MVP, sino en reforzar su confiabilidad. El proyecto mantuvo la planificación original, pero la versión final quedó más sólida gracias a la incorporación de pruebas automatizadas que complementan las validaciones manuales realizadas durante el desarrollo.

10. Evidencias del producto final

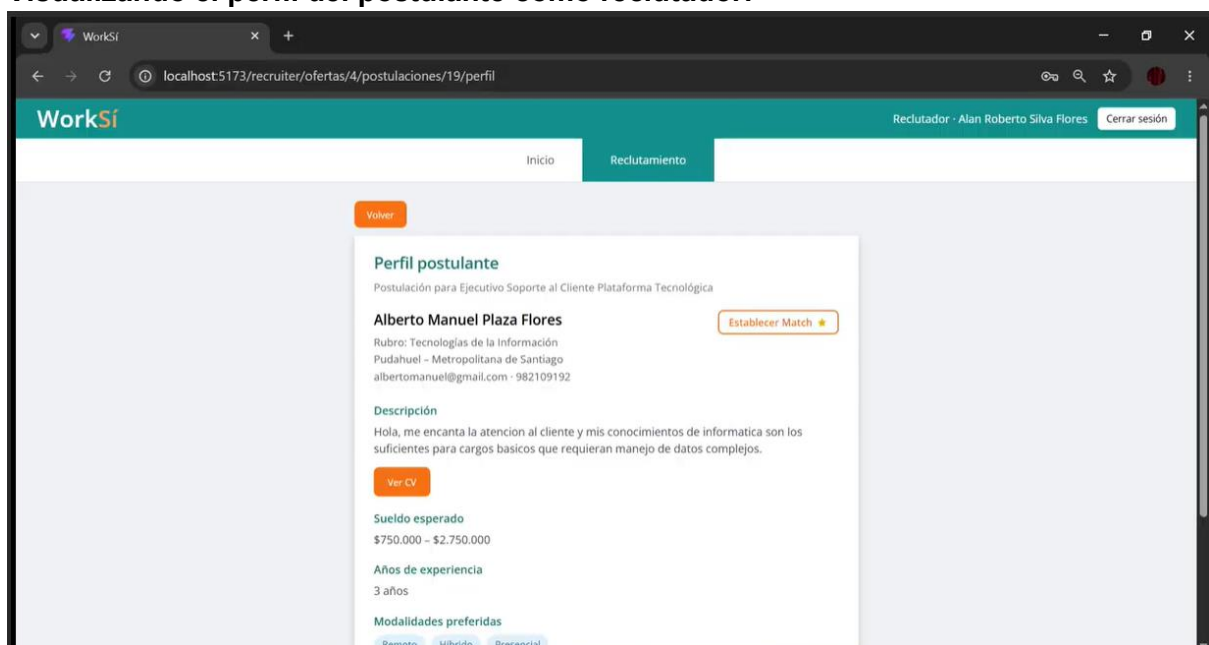
Página principal para empresas WEB:



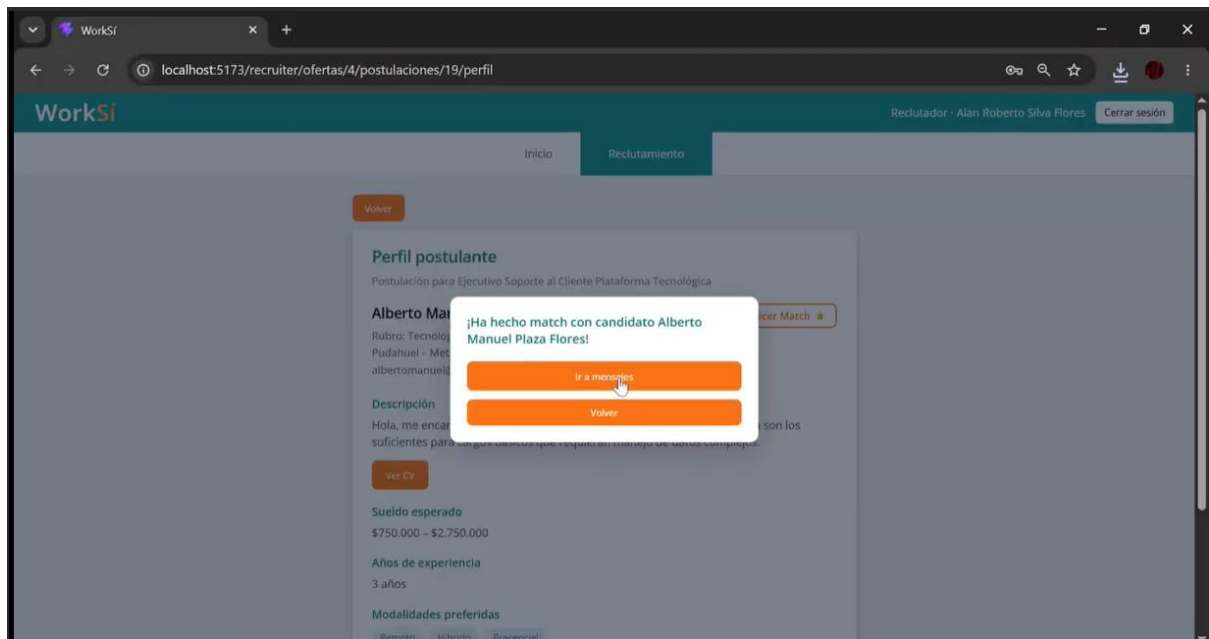
Publicación laboral del reclutador:



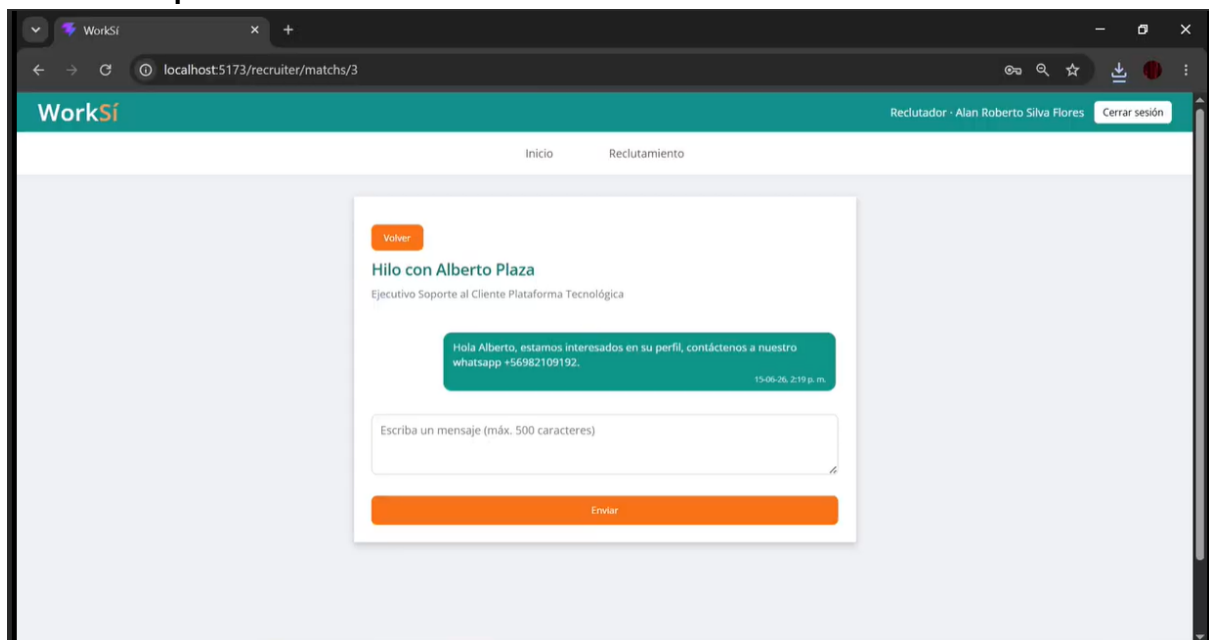
Visualizando el perfil del postulante como reclutador:



Haciendo match con el postulante:



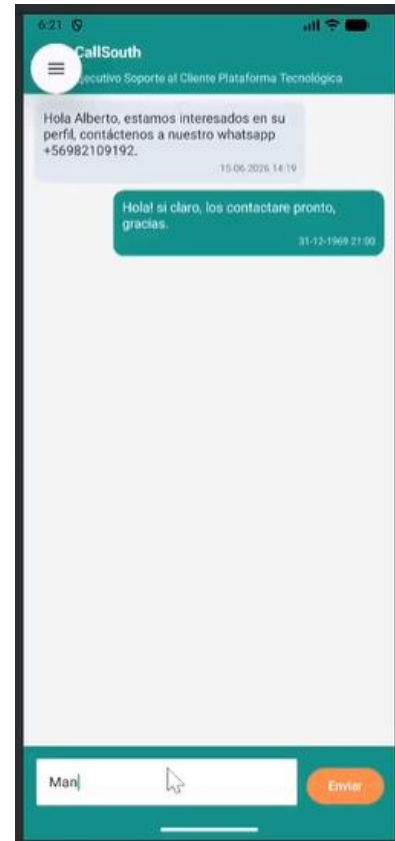
Chat con el postulante:



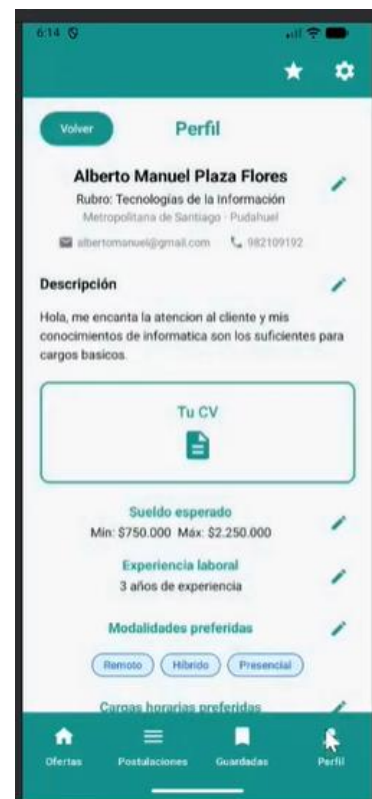
Match desde la App postulante



Chat como postulante:

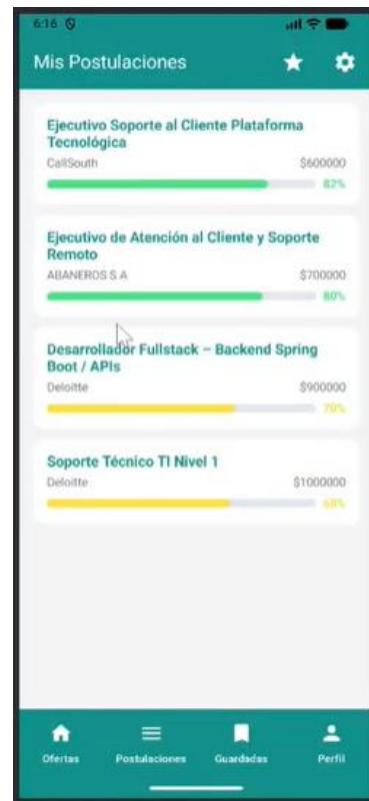


Visualizando oferta como postulante:



Matching del postulante:

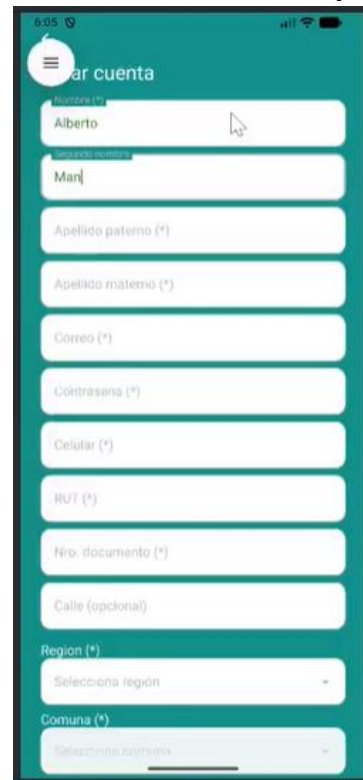
Perfil del postulante:



Postulación exitosa:



Creación de cuenta de postulante:



Apartado de Mis Postulaciones:

Creación de cuenta de postulante 2:

Experiencias

Presentación personal (opcional)

Hola, me encanta la atención al cliente

Renta esperada (opcional)

Ajusta los dos puntos en la línea. Si no la mueves, no se guarda rango de renta.

Mínima: \$1.000.000 Máxima: \$2.500.000

Atención: las modalidades, cargas horarias y años de experiencia declarados se consideran en el porcentaje de matching.

Años de experiencia laboral (*)

Indica cuántos años de experiencia profesional tienes (0 si eres recién egresado).

0 años

Modalidades preferidas (*) (al menos una)

Remoto Híbrido Presencial

Cargas horarias preferidas (*) (al menos una)

Jornada completa Part time Otro

Siguiendo

Visualización de CV en la App

Tu CV

Vicente Aravena Pavez

Perfil

Desarrollador de software con experiencia en desarrollo web y móvil. Especializado en React Native y JavaScript. Conocimiento en Node.js, Express y MongoDB. Experiencia en el desarrollo de aplicaciones para iOS y Android.

Experiencia laboral

Compañía: Universidad de los Andes en Chile

Desarrollador de software en el departamento de desarrollo web. Responsable del desarrollo de la plataforma de gestión de cursos y exámenes. Colaboré en el desarrollo de la aplicación móvil para la gestión de la biblioteca.

Formación

Universidad de los Andes en Chile

Graduado en Ingeniería en Computación. Título de Ingeniero en Computación. Año de graduación: 2018.

Referencias

Descargar CV **Cambiar CV**

Creando registro de empresa como administrador:

Registrar empresa

RUT (sin puntos, con guión)

12.345.678-5

Nombre comercial: CallSouth

Razón social:

Teléfono de contacto:

Correo corporativo:

Dirección:

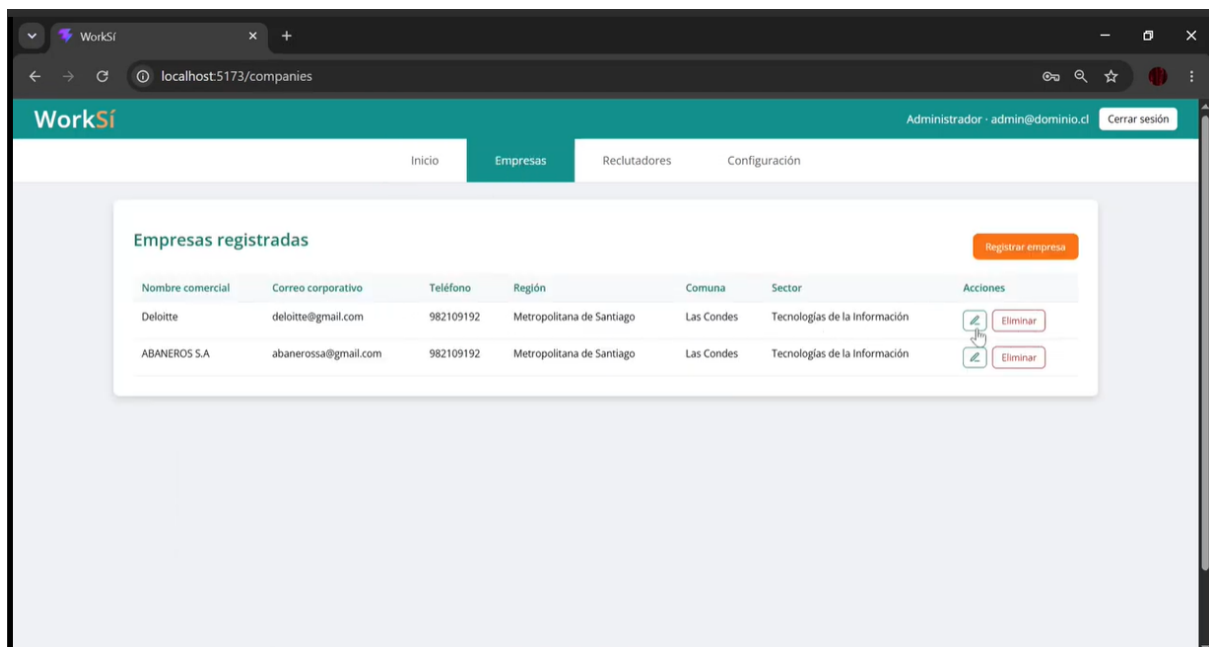
Región: Seleccione

Comuna: Seleccione

Sector o ámbito: Seleccione

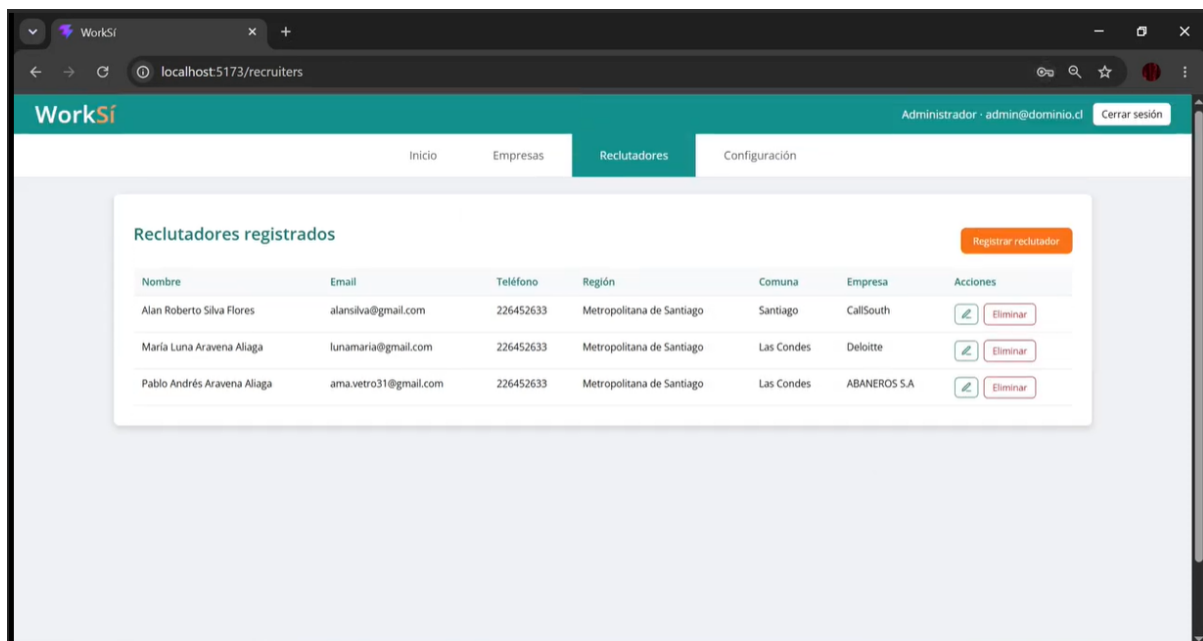
N° trabajadores (aprox.):

Visualización de las empresas creadas como administrador:



Creando a reclutador como administrador:

Visualización de reclutadores como administrador:



11. Documentación de control de versiones

El control de versiones del proyecto WorkSí se realizó mediante Git y GitHub, utilizando el repositorio como respaldo principal del avance técnico, documental y funcional del sistema. A través de este repositorio se registraron los cambios aplicados en el backend, frontend web, aplicación móvil, servicio de inteligencia artificial, configuraciones de ambiente y documentación asociada al MVP.

El trabajo se organizó principalmente mediante commits asociados a sprints, lo que permitió mantener una trazabilidad clara del avance del proyecto. En el historial del repositorio se observan registros desde Sprint 1 hasta Sprint 10, etapa que representaba el alcance base considerado inicialmente para el MVP. Sin embargo, el desarrollo continuó con avances posteriores, incluyendo actualización documental previa al Sprint 11 y cierre del Sprint 11, lo que permitió entregar una versión final más completa que la inicialmente planificada.

Dentro de esta versión más avanzada del MVP se incorporó una mejora relevante en la validación técnica del proyecto: la implementación de pruebas automatizadas. Estas pruebas se organizaron en tres grupos principales: tests unitarios Java para validar servicios y reglas de cálculo de forma aislada, tests de integración Java para comprobar el flujo entre backend, base de datos y servicio de IA, y tests Python para validar el servicio ai-service, incluyendo health, validaciones, matching semántico y limpieza de datos sensibles.

Esta evolución demuestra que el control de versiones no solo permitió registrar avances de funcionalidades, sino también evidenciar mejoras en la calidad del software. La versión final que se entrega corresponde a un MVP funcional y más fortalecido, ya que además de las pruebas manuales realizadas durante el desarrollo, se agregaron pruebas automatizadas que respaldan componentes críticos del sistema, especialmente el cálculo de compatibilidad y el servicio de inteligencia artificial.

En síntesis, GitHub permitió mantener un registro ordenado del avance del proyecto, respaldar los cambios realizados por sprint y demostrar que WorkSí evolucionó desde un MVP base hasta una versión final más consolidada para la entrega.

12. Configuraciones solicitadas en entregas anteriores

Durante la revisión del proyecto, el director realizó una observación importante relacionada con el sistema de matching. La inquietud surgió a partir de que, si el sistema solo mostraba una barra general de porcentaje, el reclutador podía no tener claridad sobre qué aspectos estaban influyendo en el resultado final. En otras palabras, un porcentaje único de compatibilidad podía ser útil como resumen, pero no explicaba si el candidato coincidía principalmente por experiencia, modalidad, carga horaria, título del cargo o descripción de la oferta.

Esta observación fue relevante porque, en un proceso real de selección, no todos los criterios tienen el mismo valor para un reclutador. Por ejemplo, para una empresa puede ser más importante que el candidato tenga los años de experiencia solicitados, mientras que para otra puede pesar más la modalidad de trabajo o el contenido técnico de su currículum. Por esta razón, se decidió mejorar la forma en que el sistema comunica el resultado del matching, entregando un desglose por dimensiones y no únicamente un porcentaje general.

Respuesta aplicada en el MVP

La respuesta implementada en WorkSí fue incorporar una vista de Detalle de Score, donde el porcentaje de match se divide en cinco dimensiones principales: descripción de la oferta, título de la oferta, modalidad, carga horaria y años de experiencia. De esta forma, el reclutador puede visualizar no solo el score final, sino también qué partes del perfil del candidato están más alineadas con la oferta publicada.

Es importante aclarar que esta mejora no corresponde todavía a un sistema de ponderaciones configurables por empresa o reclutador. Es decir, el MVP aún no permite que el reclutador defina manualmente que una dimensión valga más que otra. Sin embargo, sí entrega una solución más transparente y explicativa, ya que permite observar el comportamiento de cada criterio por separado. Esto responde directamente a la observación recibida, porque convierte el porcentaje general en una lectura más detallada y útil para la toma de decisiones.

Respaldo técnico en backend

A nivel de backend, esta mejora se encuentra implementada en el servicio ProductMatchService. En este servicio se calcula el porcentaje de compatibilidad considerando cinco dimensiones distintas. Primero se obtienen puntajes para la descripción y el título de la oferta mediante el servicio de inteligencia artificial. Luego se calculan puntajes adicionales según la modalidad preferida del candidato, la carga horaria preferida y los años de experiencia.

```java

```

double descriptionScore = toDimensionScore(descAi.get().getScore());
double titleScore = toDimensionScore(titleAi.get().getScore());
double modalityScore = preferenceDimensionScore(candidateUserId,
job.getModality().name(), true);
double workloadScore = preferenceDimensionScore(candidateUserId,
job.getWorkload().name(), false);
int candidateYears = resolveCandidateYears(candidateUserId);
double experienceScore =
 ExperienceScoreUtil.compute(candidateYears, job.getYearsExperienceRequired());
...

```

Posteriormente, el sistema calcula el score final como el promedio de estas cinco dimensiones. Esto permite mantener una puntuación general, pero basada en componentes internos claramente identificables:

```

```java
double finalScore =
    round2(
        clamp(
            (descriptionScore
              + titleScore
              + modalityScore
              + workloadScore
              + experienceScore)
            / 5.0,
            0.0,
            100.0));
...

```

Además, el backend construye un objeto MatchBreakdownResponse con el resultado final y cada uno de los puntajes individuales. Esto permite que el frontend no reciba solo el porcentaje total, sino también el detalle que será mostrado visualmente al reclutador:

```

```java
MatchBreakdownResponse breakdown =
 new MatchBreakdownResponse(
 finalScore,
 descriptionScore,
 titleScore,
 modalityScore,
 workloadScore,
 experienceScore);
...

```

## Estructura de respuesta enviada al frontend



El objeto MatchBreakdownResponse define explícitamente los campos que se envían como respuesta. Cada propiedad se encuentra preparada para ser consumida como JSON por la plataforma web:

```
```java
@JsonProperty("final_score")
private final Double finalScore;

@JsonProperty("description_score")
private final Double descriptionScore;

@JsonProperty("title_score")
private final Double titleScore;

@JsonProperty("modality_score")
private final Double modalityScore;

@JsonProperty("workload_score")
private final Double workloadScore;

@JsonProperty("experience_score")
private final Double experienceScore;
```
```

Con esta estructura, el backend entrega un resultado más completo. Por ejemplo, una postulación puede tener un score final de 69%, pero al revisar el desglose se puede observar que obtuvo 100% en modalidad y carga horaria, 50% en años de experiencia, 62% en descripción de la oferta y 33% en título de la oferta. Esta información permite que el reclutador interprete mejor por qué el resultado final tiene ese valor.

## Respaldo técnico en frontend

En la plataforma web, la vista RecruiterApplicationView define las dimensiones que se mostrarán al reclutador. Estas dimensiones corresponden directamente a los campos enviados por el backend:

```
```javascript
const SCORE_DIMENSIONS = [
  { key: "description_score", label: "Descripción de la oferta" },
  { key: "title_score", label: "Título de la oferta" },
  { key: "modality_score", label: "Modalidad" },
  { key: "workload_score", label: "Carga horaria" },
  { key: "experience_score", label: "Años de experiencia" },
];
```
```

La misma vista obtiene el desglose desde la postulación mediante `match_breakdown` y utiliza `final_score` cuando este se encuentra disponible. Esto permite mostrar el score final y, al mismo tiempo, desplegar las dimensiones individuales:

```
````javascript
const breakdown = item?.match_breakdown;
const finalScore =
  breakdown?.final_score !== null
    ? breakdown.final_score
    : item?.match_score;
````
```

Finalmente, el frontend recorre cada dimensión y genera una barra visual para representar el porcentaje obtenido. Esto es lo que se observa en la pantalla de Detalle de Score, donde cada aspecto aparece con su nombre, porcentaje y barra correspondiente:

```
````javascript
{SCORE_DIMENSIONS.map(({ key, label }) => {
  const val = breakdown[key];
  const pct = val !== null ? Math.round(val) : 0;
  return (
    <div key={key} className="recruiter-score-detail-dimension">
      <div className="recruiter-score-detail-dimension-head">
        <span>{label}</span>
        <strong>{pct}%</strong>
      </div>
      <div className="score-bar-track recruiter-score-detail-bar">
        <div
          className="score-bar-fill"
          style={{
            width: `${Math.min(100, Math.max(0, pct))}%`,
          }}
        />
      </div>
    </div>
  );
})}
````
```

### **Relación con la observación recibida**

Gracias a esta implementación, el sistema ya no depende únicamente de una barra general de match. El porcentaje final sigue existiendo como resumen, pero ahora se complementa con una explicación visual por dimensiones. Esto permite que el reclutador analice con mayor claridad si un candidato es fuerte en aspectos técnicos, si coincide con las condiciones del cargo o si presenta una brecha específica en experiencia.

En términos del alcance del MVP, esta solución representa una mejora importante de transparencia y explicabilidad. Si bien una versión futura podría permitir que cada reclutador

configure pesos personalizados para cada dimensión, la versión actual ya entrega una respuesta concreta a la observación realizada, ya que permite entender el resultado del matching de forma más detallada y fundamentada.

En síntesis, la configuración solicitada fue abordada mediante una mejora funcional y visual del sistema de matching. El backend calcula y expone el desglose por dimensiones, mientras que el frontend lo presenta al reclutador mediante barras de progreso individuales. Esto fortalece la utilidad del sistema, ya que transforma el score de compatibilidad en una herramienta más clara para apoyar la toma de decisiones dentro del proceso de selección.

### **13. Documento de aceptación del cliente**

En el caso del proyecto WorkSí, no se cuenta con un documento de aceptación de cliente en el formato tradicional, debido a que el sistema no fue desarrollado para una empresa contratante específica ni para un cliente externo único que solicitara formalmente el producto. A diferencia de un proyecto encargado por una organización, WorkSí fue planteado como una solución propia del equipo, con una visión de producto digital orientado al mercado laboral.

El proyecto se define como una propuesta tipo startup, cuyo objetivo es desarrollar una plataforma que pueda ser utilizada en el futuro tanto por empresas que buscan optimizar sus procesos de reclutamiento como por personas que desean encontrar oportunidades laborales desde una aplicación móvil. Por esta razón, la aceptación del producto no depende de la firma de un cliente puntual, sino de la validación progresiva del MVP, de su funcionamiento técnico y de la utilidad que pueda demostrar frente a usuarios reales en una eventual etapa de lanzamiento.

Durante esta evaluación, la validación del producto se realizó principalmente desde el contexto académico y técnico. Esto significa que el equipo comprobó que el MVP respondiera a la problemática planteada, que los flujos principales funcionaran correctamente y que las funcionalidades desarrolladas permitieran representar una primera versión utilizable de la plataforma. En este proceso fueron importantes las pruebas manuales, las pruebas mediante Postman, la ejecución de la plataforma web y aplicación móvil, además de las pruebas automatizadas incorporadas en la versión final del proyecto.

Desde esta perspectiva, el producto puede considerarse aceptado internamente como MVP funcional, ya que cumple con los elementos principales definidos para esta etapa: registro y autenticación de usuarios, gestión de empresas y reclutadores, publicación de ofertas laborales, interacción del candidato con las vacantes, cálculo de compatibilidad mediante inteligencia artificial y visualización detallada del score de match. Estos elementos permiten demostrar que la solución propuesta puede ser utilizada como base para una versión más robusta y preparada para usuarios reales.

En una etapa posterior, cuando WorkSí avance hacia una publicación o lanzamiento más formal, la aceptación del producto debería realizarse mediante mecanismos propios de una startup tecnológica. Entre ellos se podrían considerar pruebas piloto con empresas, uso controlado por candidatos, encuestas de satisfacción, métricas de uso, retroalimentación de reclutadores, análisis de postulaciones realizadas y validación del valor entregado por el

sistema de matching. De esta forma, la aceptación futura estaría relacionada con la adopción del producto y con su capacidad de resolver necesidades reales del mercado.

En síntesis, aunque no existe un documento formal de aceptación firmado por un cliente externo, el proyecto cuenta con una validación académica, funcional y técnica del MVP desarrollado. Esta validación permite respaldar que WorkSí cumple con el alcance definido para la entrega final y que se encuentra en condiciones de ser presentado como una primera versión de producto, con proyección a mejoras, pruebas con usuarios reales y eventual lanzamiento al mercado.

## **14. Conclusión**

El desarrollo del proyecto WorkSí permitió consolidar una solución tecnológica orientada a mejorar los procesos de reclutamiento laboral mediante una plataforma compuesta por aplicación móvil, plataforma web, backend, base de datos y servicio de inteligencia artificial. A lo largo del proyecto se logró avanzar desde una idea inicial hasta un MVP funcional, capaz de representar los flujos principales de candidatos, reclutadores y administradores, incorporando registro de usuarios, gestión de ofertas, carga de currículum, postulación y cálculo de compatibilidad entre candidato y oferta laboral.

Uno de los aspectos más relevantes del proyecto fue la incorporación del matching laboral, ya que este componente permite entregar apoyo al proceso de selección mediante un porcentaje de compatibilidad. Además, a partir de la observación recibida respecto a que una sola barra de porcentaje podía ser insuficiente para la toma de decisiones, se incorporó un desglose del score por dimensiones. Esto permitió que el reclutador pudiera visualizar no solo el resultado final, sino también aspectos específicos como descripción de la oferta, título, modalidad, carga horaria y años de experiencia, entregando mayor transparencia al proceso.

En relación con la validación del producto, se debe considerar que WorkSí no fue desarrollado para un cliente externo único, sino como una propuesta propia con enfoque de startup. Por esta razón, no existe un documento formal de aceptación de cliente, ya que el producto está pensado para ser lanzado posteriormente al mercado como una aplicación y plataforma que puedan ser utilizadas por empresas y personas en búsqueda de empleo. En esta etapa, la validación se centró en el cumplimiento del MVP, la revisión técnica, las pruebas funcionales y la retroalimentación académica recibida durante el avance del proyecto.

Finalmente, se puede concluir que el proyecto cumple con los objetivos principales planteados, ya que permite representar una solución funcional frente a la problemática identificada en los procesos tradicionales de reclutamiento. Si bien aún existen oportunidades de mejora para una futura versión comercial, como pruebas con usuarios reales, ajustes de experiencia de usuario, métricas de uso y validación en mercado, el MVP desarrollado demuestra una base sólida para seguir evolucionando. WorkSí logró integrar distintas tecnologías y componentes en una solución coherente, funcional y proyectable a futuro.

## **15. Lecciones aprendidas**

Durante el desarrollo de WorkSí, una de las principales lecciones aprendidas fue la importancia de organizar el trabajo de manera progresiva. Al inicio del proyecto existía una visión general de la solución, pero a medida que avanzaron los sprints fue necesario ordenar prioridades, dividir responsabilidades y ajustar el alcance según el tiempo disponible. Esto permitió comprender que un proyecto de software no se construye completo desde el primer momento, sino que requiere planificación, revisión constante y avance por etapas.

También se aprendió la importancia de trabajar de forma colaborativa y mantener una comunicación constante dentro del equipo. El proyecto integró distintas áreas, como backend, frontend web, aplicación móvil, base de datos, inteligencia artificial, pruebas y documentación. Esto obligó a coordinar tareas, revisar avances de otros integrantes y entender cómo cada componente afectaba al resto del sistema. A medida que el proyecto fue creciendo, la organización del equipo se volvió fundamental para evitar errores, duplicidad de trabajo o problemas de integración.

Otra lección importante fue comprender que las pruebas no solo sirven para detectar errores al final, sino también para confirmar que el desarrollo va por buen camino. En una primera etapa, las validaciones se realizaron principalmente mediante Postman y ejecución manual de la web y la app móvil. Posteriormente, con la incorporación de tests unitarios, tests de integración y pruebas sobre el servicio de IA, se logró reforzar la confianza técnica del proyecto. Esto permitió entender la diferencia entre probar visualmente una funcionalidad y validar internamente que las reglas, cálculos e integraciones funcionen correctamente.

Además, el proyecto permitió aprender a recibir observaciones y transformarlas en mejoras concretas. Un ejemplo de esto fue la observación relacionada con el porcentaje de matching, donde se identificó que un único valor podía no ser suficiente para explicar la compatibilidad entre candidato y oferta. A partir de esto, se incorporó el detalle del score por dimensiones, mejorando la explicabilidad del sistema y acercándolo más a una herramienta útil para reclutadores.

Finalmente, trabajar en WorkSí permitió comprender mejor cómo se construye un producto con proyección real. Al no tratarse de un sistema solicitado por un cliente específico, sino de una propuesta propia tipo startup, fue necesario pensar no solo en cumplir con una entrega académica, sino también en cómo podría evolucionar el producto en el futuro. Esto permitió valorar aspectos como la experiencia de usuario, la escalabilidad, la validación con usuarios reales y la necesidad de seguir iterando el MVP antes de un eventual lanzamiento al mercado.